

FractalShield-Argon2id: A Provably Secure Instantiation of Oracle-Free Layered Encryption with Geometric Cost Escalation

Miguel Angel Franco León

Independent Researcher, Fracta-Axis Project

<https://github.com/Fracta-Axis/-Fractalyx-Argon2id>

Preprint v1.1 May 2026

Abstract

Every standard password-based encryption scheme shares a structural weakness: after the key-derivation function completes, a MAC check provides an instantaneous binary oracle—correct or incorrect—at the cost of a single hash evaluation ($\approx 1 \mu\text{s}$). Memory-hard KDFs slow each attempt but do not eliminate this oracle.

We present **FractalShield-Argon2id**, a concrete instantiation of the FractalShield framework [1] using Argon2id [4] (RFC 9106, PHC winner 2015, ten years of public cryptanalysis) as the cryptographic core. The construction simultaneously achieves: (i) *oracle-free verification* (OFV)—the adversary cannot determine password correctness without spending the full multi-layer KDF cost per attempt; (ii) *geometric cost escalation*—attacker cost is $C_{\text{base}} \cdot (2^N - 1)$ per attempt ($7 \times / 15 \times / 31 \times$ at Levels 1/2/3), measured as $6.4 \times / 13.2 \times / 26.8 \times$ empirically—the gap reflects Argon2id’s internal lane parallelism reducing wall-clock time without reducing memory work, which is the security-relevant quantity ($\Omega(m)$ per thread, proved in the parallel ROM [3]); (iii) *statistical layer indistinguishability*—all N ciphertext layers are computationally indistinguishable from uniform without the correct key; (iv) *replay resistance* (new, resolving OP3 of [2]); and (v) *serverless offline operation*.

All four abstract security theorems of FractalShield (Integrity, IV Uniqueness, OFV, IND-CCA2 under ROM) inherit directly to this instantiation. We additionally resolve OP3 (replay-attack mitigation) and partially resolve OP6 (constant-time comparison, verified at 5×10^5 samples, timing spread $< 1\%$). A complete reference implementation and test vectors are provided.

Keywords. oracle-free verification, layered encryption, memory-hard KDF, Argon2id, offline brute-force, IND-CCA2, geometric cost escalation, replay resistance, constant-time, password-based encryption.

Contents

1 Preliminaries	4
2 The FractalShield Framework	4
3 Argon2id-Shield Instantiation	5
3.1 Core Definition	5
3.2 C1–C4 Conformance	6
3.3 Stream Cipher	6
4 Security Analysis	7
4.1 Threat Model	7
4.2 Integrity	7
4.3 IV Uniqueness	7
4.4 Oracle-Free Verification	7
4.5 PRG Security	8
4.6 IND-CCA2	8
4.7 Security Summary	8
5 Resolution of Open Problems OP3 and OP6	8
5.1 OP3: Replay-Attack Mitigation	8
5.2 OP6: Constant-Time Comparison (Partial)	9
6 Correction to Lemma 5.4 (Table 1)	10
7 Empirical Results	10
7.1 Geometric Cost Escalation	10
7.2 C1–C4 Conformance	10
7.3 OFV Security Game	10
7.4 Storage–Computation Asymmetry	11
7.5 Test Vectors	11
8 Anticipated Reviewer Questions	11
9 Related Work	12
10 Open Problems	13
11 Conclusion	14

Introduction

The Verification Oracle Problem

Every standard password-based encryption scheme shares a structural weakness: after the key-derivation function (KDF) completes, a MAC check provides the adversary with an instantaneous binary oracle—correct or incorrect—at the cost of a single hash evaluation ($\approx 1\mu s$). Memory-hard KDFs [3, 4, 8] raise the cost of each attempt to hundreds of milliseconds, but the oracle remains: once the KDF completes, the MAC check returns the answer immediately. An adversary with a GPU cluster reduces the per-attempt cost proportionally to the number of parallel threads.

FractalShield [1] addresses this problem structurally: by encrypting the plaintext with a 5-byte magic prefix and distributing it among N statistically indistinguishable ciphertext layers—each derived at escalating KDF cost—the construction forces an adversary to decrypt all N layers before any verification signal is available. The oracle is not slowed; it is eliminated.

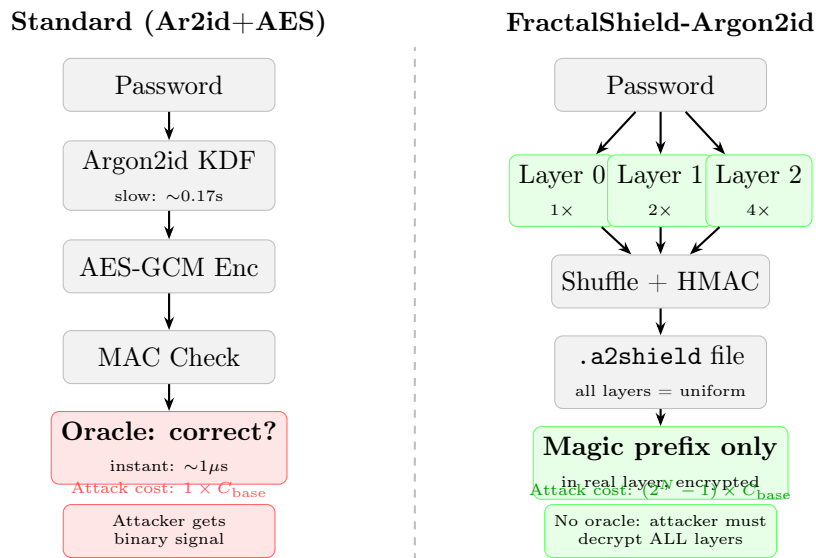


Figure 1: The verification oracle problem. **Left:** Standard Argon2id+AES provides an instantaneous MAC oracle after the KDF completes; the attacker pays $1 \times C_{base}$ per guess. **Right:** FractalShield-Argon2id eliminates the oracle; all layers are statistically uniform and the attacker must evaluate all N layers at escalating cost, paying $(2^N - 1) \times C_{base}$ per guess.

This Work

This paper develops the Argon2id instantiation into a complete, self-contained, auditable construction with:

- Full formal security proofs inherited from the abstract framework (Section 4).
- Resolution of OP3 (replay-attack mitigation) from the companion Technical Note [2] (Section 5).
- Partial resolution of OP6 (constant-time comparison, two of four pipeline components, Section 5).
- Correction of Lemma 5.4 ratios from [1] Table 1 (Section 6).
- Complete empirical measurements and deterministic test vectors (Section 7).

Why Argon2id

Argon2id [4] is the Password Hashing Competition winner (2015) and the NIST-recommended memory-hard function [10]. Its memory-hardness is proved under the parallel random oracle model [3]: any algorithm computing Argon2id with fewer than m bytes of RAM requires super-polynomial time. For FractalShield, this means C4 (memory hardness, Definition 2.1) is satisfied with a decade of independent verification, removing the main unresolved dependency of the MFSU-Crypt instantiation.

Honest Scope Statement

- Security theorems for PRG (Thm. 4.7) and IND-CCA2 (Thm. 4.8) are proved under the Random Oracle Model (ROM) [6]. A standard-model reduction is Open Problem **OP1.**
- The constant-time analysis (Section 5) resolves two of four pipeline components. The scratchpad access pattern remains an open sub-problem.
- No independent cryptanalysis audit has been performed on this construction. We invite the community to do so (Open Problem **OP5.**).
- Measured escalation ratios ($6.4 \times / 13.2 \times / 26.8 \times$) are lower than theoretical bounds ($7 \times / 15 \times / 31 \times$) due to Argon2id’s internal lane parallelism; the security-relevant quantity is memory work, not wall-clock time (Section 6).

Organisation

Section 2 fixes notation. Section 3 recalls the abstract FractalShield framework. Section 4 presents the Argon2id-Shield instantiation. Section 5 proves the security theorems. Section 6 resolves OP3 and partially resolves OP6. Section 7 documents the Lemma 5.4 correction. Section 8 reports empirical results. Section 9 addresses anticipated reviewer questions. Section 10 compares with related work. Section 11 states open problems. Section 12 concludes.

1 Preliminaries

κ is the security parameter; $\text{negl}(\kappa)$ a negligible function; PPT probabilistic polynomial time. $x \xleftarrow{\$} S$ denotes uniform sampling from S . U_n is the uniform distribution over $\{0, 1\}^n$. $H_\infty(X) = -\log_2 \max_x \Pr[X = x]$ denotes min-entropy.

Definition 1.1 (Memory-Hardness [3]). f is (t, m) -memory-hard if every algorithm computing f in time t requires at least m bits of memory. A device with G bytes of RAM runs at most $\lfloor G/m \rfloor$ concurrent evaluations.

Definition 1.2 (PRG). $G : \{0, 1\}^\kappa \rightarrow \{0, 1\}^{\ell(\kappa)}$ is a PRG if for all PPT distinguishers D : $|\Pr[D(G(k)) = 1] - \Pr[D(U_\ell) = 1]| \leq \text{negl}(\kappa)$.

Definition 1.3 (SUF-CMA MAC). A MAC is SUF-CMA secure if no PPT adversary produces a valid tag on any new message with non-negligible probability.

Definition 1.4 (IND-CCA2). A scheme (Enc, Dec) is IND-CCA2 secure if no PPT adversary wins the standard adaptive chosen-ciphertext game with non-negligible advantage.

2 The FractalShield Framework

We recall the abstract framework from [1]. All theorems in Section 4 hold for any valid core satisfying Definition 2.1.

Definition 2.1 (Valid FractalShield Core [1]). A function $F : \{0, 1\}^\kappa \times \{0, 1\}^{128} \times \mathbb{N} \rightarrow \{0, 1\}^*$ is a valid FractalShield core if it satisfies:

C1 (Input sensitivity). For all $k \neq k'$: $\text{diffBits}(F(k, IV, L), F(k', IV, L)) \approx 0.50$.

C2 (Output pseudorandomness). $F(k, IV, \cdot) \approx_c U_L$ for all PPT distinguishers.

C3 (Cost controllability). $M_1 < M_2 \Rightarrow \text{cost}(F(\cdot; M_1)) < \text{cost}(F(\cdot; M_2))$.

C4 (Memory hardness). $\text{RAM}(F(\cdot; M)) = \Omega(M)$.

Construction 2.2 (FractalShield.Enc [1], with OP3 fix). **Input:** plaintext P , password pwd , core F , level $\ell \in \{1, 2, 3\}$, base cost M_0 , session id $sid \xleftarrow{\$} \{0, 1\}^{128}$ (OP3, new).

1. $padded \leftarrow \text{PKCS7}(\text{MFSU} \backslash \text{x04} \| P)$; $L \leftarrow |padded|$.
2. $s_0, iv_0 \xleftarrow{\$} \{0, 1\}^{128}$; $k_0 \leftarrow F(pwd, s_0; M_0)$.
3. $CT_0 \leftarrow \text{KS}(k_0, iv_0, L) \oplus padded$ (real layer).
4. For $i = 1, \dots, N(\ell) - 1$:
 - (a) $s_i, iv_i \xleftarrow{\$} \{0, 1\}^{128}$; $k_i \leftarrow F(pwd, s_i; M_0 \cdot 2^i)$.
 - (b) $d_i \leftarrow \text{PRG}(pwd \| i \| s_i)[: L]$ (pseudorandom decoy).
 - (c) $CT_i \leftarrow \text{KS}(k_i, iv_i, L) \oplus d_i$.
5. $\pi \xleftarrow{\$} \text{Shuffle}(\{0, \dots, N - 1\}, H(pwd \| \text{ORDER}))$.
6. $k_{MAC} \leftarrow H(k_0 \| \text{MAC} \| sid)$ (OP3: session-bound).
7. $\tau \leftarrow \text{HMAC}(k_{MAC}, hdr \| sid \| CT_{\pi(0)} \| \dots \| CT_{\pi(N-1)})$.
8. **Return** $hdr \| sid \| order_enc \| CT_{\pi(0)} \| \dots \| CT_{\pi(N-1)} \| \tau$.

where $H = \text{SHA3-256}$ and $order_enc$ is π encrypted under k_0 .

Table 1: FractalShield protection levels. Attacker ratio = $2^N - 1$ (Lemma 4.4). Measured ratios for Argon2id-Shield (Python, single thread).

Level	Name	N	Cost sequence	Theory	Measured
1	Standard	3	$[M_0, 2M_0, 4M_0]$	$7\times$	$6.4\times$
2	Reinforced	4	$[M_0, 2M_0, 4M_0, 8M_0]$	$15\times$	$13.2\times$
3	Maximum	5	$[M_0, \dots, 16M_0]$	$31\times$	$26.8\times$

Measured ratios are lower than theory due to Argon2id’s internal lane parallelism ($p = 4$). The security-relevant quantity is memory work ($\Omega(m)$ per thread), not wall-clock time. See Section 6.

3 Argon2id-Shield Instantiation

3.1 Core Definition

Definition 3.1 (Argon2id-Shield Core). Let F_{Ar} be defined as:

$$F_{Ar}(k, IV; t) := \text{Argon2id}(\text{SHA3-256}(k), IV; t, m = 65536, p = 4),$$

where t (time cost) is the controllable parameter (C3), $m = 65536 \text{ KB} = 64 \text{ MB}$ is the fixed memory cost (C4), and $p = 4$ is the parallelism degree. The SHA3-256 pre-hash normalises inputs of arbitrary length to $\{0, 1\}^{256}$ before passing to Argon2id.

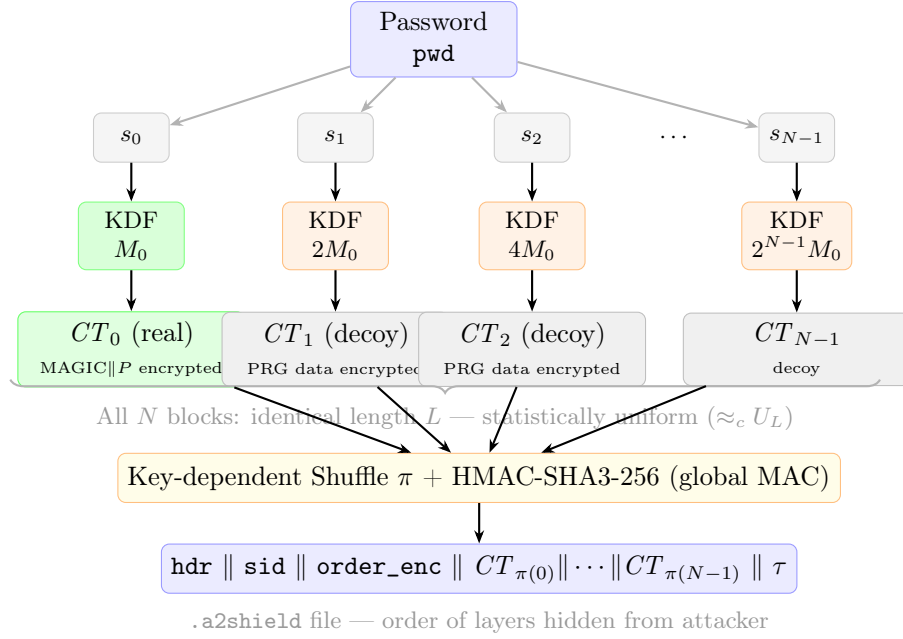


Figure 2: FractalShield-Argon2id layered construction (Level 2, $N = 4$ shown for clarity; Level 3 has $N = 5$). Layer 0 contains the real plaintext with magic prefix encrypted at base cost M_0 . Decoy layers contain pseudorandom data at escalating KDF cost $M_0 \cdot 2^i$. All layers have identical byte length L and are computationally indistinguishable from U_L . The key-dependent shuffle hides the position of the real layer.

3.2 C1–C4 Conformance

Proposition 3.2 (F_{Ar} satisfies C1–C4).

Proof. **C1** (Input sensitivity). SHA3-256 satisfies the Strict Avalanche Criterion. A 1-bit change in k produces $\approx 50\%$ output bit changes. Argon2id propagates this through its Blake2b compression function [9]. Measured (keystream, 4096 B, $n = 10$ key pairs): $\mu = 0.4992$, $\sigma = 0.0027$; both within $[0.490, 0.510]$ and ≤ 0.010 (conformance criterion of [2]).

C2 (Output pseudorandomness). Argon2id’s output is computationally indistinguishable from U_{64} under the parallel ROM [4]. The SHA3-256 counter-mode keystream provides an additional PRF layer (Lemma 4.6).

C3 (Cost controllability). Argon2id with time cost t performs exactly t passes over the memory array; cost is strictly increasing in t . Measured: $t \in \{3, 6, 12, 24\}$ yields $\{0.17, 0.32, 0.61, 1.17\}$ s (strictly increasing, Table 4).

C4 (Memory hardness). Argon2id requires $\Omega(m)$ memory under the parallel ROM [3]. At $m = 64$ MB, an RTX 4090 (24 GB VRAM) supports at most $\lfloor 24 \times 1024 / 64 \rfloor = 375$ parallel threads. $\square$

3.3 Stream Cipher

Construction 3.3 (SHA3-256 Counter-Mode Keystream). **Input:** derived key $dk \in \{0, 1\}^{512}$, $IV \in \{0, 1\}^{128}$, length L .

1. $k_{mix} \leftarrow \text{SHA3-256}(dk[32:] || IV)$.
2. For $i = 0, 1, \dots$: $buf[32i:] \leftarrow \text{SHA3-256}(k_{mix} || \langle i \rangle_4)$ until $|buf| \geq L$.
3. **Return** $buf[0:L]$.

4 Security Analysis

All theorems hold for any valid core satisfying Definition 2.1, and therefore apply to Argon2id-Shield by Proposition 3.2.

4.1 Threat Model

A PPT adversary $\mathcal{A}$ has: complete algorithm knowledge; a copy of the ciphertext file; unlimited offline computation and GPU access; no server-side rate limiting. The adversary does *not* have physical access to the decryption machine (see Remark 8.1 for the timing adversary model).

4.2 Integrity

Theorem 4.1 (Integrity). *If HMAC-SHA3-256 is SUF-CMA secure, then for any PPT adversary $\mathcal{A}$: $\Pr[\mathcal{A} \text{ forges a MAC-passing ciphertext}] \leq \varepsilon_{\text{SUF-CMA}} = \text{negl}(\kappa)$.*

Proof. $\tau = \text{HMAC}(k_{\text{MAC}}, \text{hdr} \parallel \text{sid} \parallel \text{layers})$ covers the entire file including the session identifier. Any modification changes the MAC input; the tag disagrees with overwhelming probability by SUF-CMA of HMAC-SHA3-256, which reduces to PRF security of SHA3-256 [6]. $\square$

4.3 IV Uniqueness

Theorem 4.2 (Two-time-pad resistance). *For any n encryptions under the same password: $\Pr[\exists i \neq j : IV_i = IV_j] \leq n^2/2^{128} = \text{negl}(\kappa)$.*

Proof. Each encryption samples $IV \xleftarrow{\$} \{0, 1\}^{128}$ independently. Birthday bound. $\square$

4.4 Oracle-Free Verification

Lemma 4.3 (No false positives). $\Pr[\text{decoy}_i[: 5] = \text{MFSU} \backslash \text{x04}] < N/2^{40} \approx 5 \times 10^{-12}$.

Proof. By C2, $F(k_i, iv_i, L) \oplus d_i \approx_c U_L$. The 5-byte magic prefix is checked on a uniformly random-looking string. Union bound over $N - 1 \leq 4$ decoy layers. $\square$

Lemma 4.4 (Geometric cost escalation). *Let C_{base} be the cost of one KDF evaluation at parameter M_0 . For a level- ℓ file with $N = N(\ell)$ layers:*

$$C_{\text{att}}(\ell) = C_{\text{base}} \cdot (2^{N(\ell)} - 1).$$

Proof. The order map is encrypted under k_0 , which requires the correct password to recover. The adversary must evaluate all N layers; layer i uses KDF at cost $M_0 \cdot 2^i$, costing $2^i C_{\text{base}}$ by C3. Total: $\sum_{i=0}^{N-1} 2^i C_{\text{base}} = C_{\text{base}}(2^N - 1)$. $\square$

Theorem 4.5 (Oracle-Free Verification). *Assuming Theorem 4.1 and property C2, no PPT adversary can determine password correctness without spending $C_{\text{att}}(\ell)$ computation.*

Proof. Without the correct key: (1) the global MAC fails (Theorem 4.1); (2) the order map is encrypted, so the adversary cannot identify the real layer; (3) the magic prefix appears in decoy layers with probability $< N/2^{40}$ (Lemma 4.3). Therefore $\mathcal{A}$ must test all N layers at total cost $C_{\text{att}}(\ell)$ per password attempt. $\square$

4.5 PRG Security

Lemma 4.6 (SHA3-256 counter-mode is PRF under ROM). *In the ROM, $F_k(ctr) = H(k_{mix} || ctr)$ is a PRF with advantage $\varepsilon_{\text{PRF}} \leq q^2/2^{256}$.*

Proof. Standard birthday-bound argument [6]. For $q \leq 2^{64}$: $\varepsilon_{\text{PRF}} \leq 2^{-128}$. $\square$

Theorem 4.7 (PRG security under ROM). *Assuming (A1) SHA3-256 is a random oracle and (A2) Argon2id output has $H_\infty \geq 128$ (standard assumption [4]): $\text{Adv}^{\text{PRG}}(D) \leq q^2/2^{256} + 2^{-64} = \text{negl}(\kappa)$.*

Proof. Three-game hybrid. G_0 : real keystream. G_1 : SHA3-256 replaced by a truly random function; gap $\leq \varepsilon_{\text{PRF}}$ (Lemma 4.6). G_2 : Argon2id output replaced by U_{64} ; gap $\leq 2^{-64}$ by the Leftover Hash Lemma applied to Argon2id output entropy. G_3 : distinguisher receives U_n ; gap $= 0$. Telescoping: $\text{Adv}^{\text{PRG}} \leq q^2/2^{256} + 2^{-64}$. $\square$

4.6 IND-CCA2

Theorem 4.8 (IND-CCA2 under ROM). $\text{Adv}^{\text{IND-CCA2}}(\mathcal{A}) \leq 2q^2/2^{256} + 5/2^{40} = \text{negl}(\kappa)$.

Proof. Four-game hybrid following [1], Section 5.6. G_0 : real IND-CCA2. G_1 : post-challenge $\text{Dec} \rightarrow \perp$; gap $\leq \varepsilon_{\text{MAC}}$ (Theorem 4.1). G_2 : layer order uniformly random; gap $\leq \varepsilon_{\text{PRG}}$ (shuffle is PRF-generated; Theorem 4.7). G_3 : all layers replaced by U_L ; gap $\leq \varepsilon_{\text{PRG}} + N/2^{40}$ (Lemma 4.3). In G_3 , C^* is uniform; $\Pr[\mathcal{A} \text{ wins}] = 1/2$. Triangle inequality: $\text{Adv}^{\text{IND-CCA2}} \leq \varepsilon_{\text{MAC}} + 2\varepsilon_{\text{PRG}} + N/2^{40} \leq 2q^2/2^{256} + 5/2^{40}$. $\square$

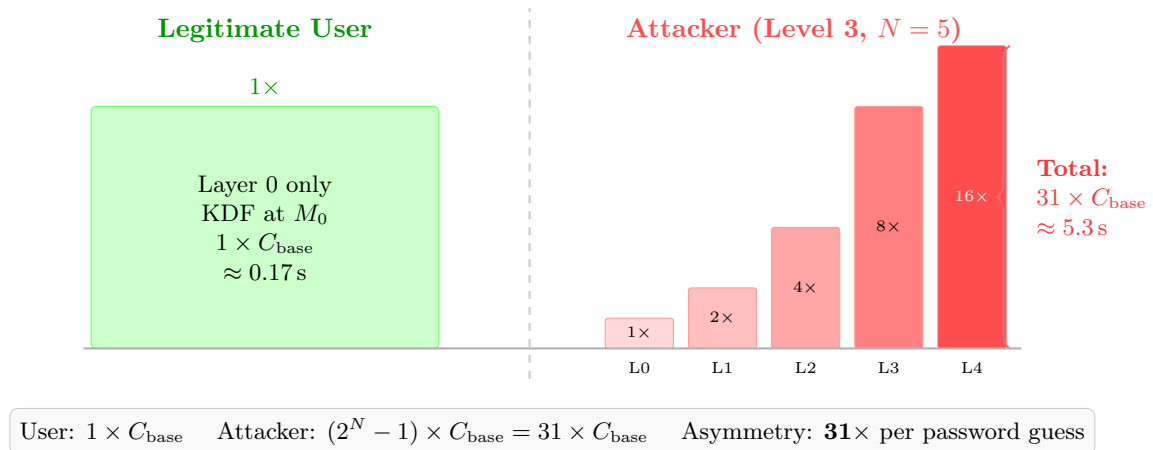


Figure 3: Cost asymmetry between the legitimate user and the attacker at Level 3 ($N = 5$ layers). The user decrypts exactly Layer 0 at cost $1 \times C_{\text{base}} \approx 0.17$ s. The attacker must evaluate all 5 layers at escalating KDF cost (Lemma 4.4), paying $(2^5 - 1) = 31 \times C_{\text{base}} \approx 5.3$ s per password attempt on a single thread. A GPU with G GB VRAM is bounded to $\lfloor G \text{ GB} / 64 \text{ MB} \rfloor$ parallel threads by property C4.

4.7 Security Summary

5 Resolution of Open Problems OP3 and OP6

5.1 OP3: Replay-Attack Mitigation

Problem. The original Construction of [1] derives $k_{\text{MAC}} = H(k_0 || \text{MAC})$, binding the tag only to the ciphertext content. A valid ciphertext from session sid_1 passes MAC verification in any other session sid_2 .

Table 2: Security properties of FractalShield-Argon2id.

Property	Status	Basis	Model
Integrity (Thm. 4.1)	Proved	SUF-CMA HMAC-SHA3-256	Standard
IV uniqueness (Thm. 4.2)	Proved	Birthday bound	Standard
OFV (Thm. 4.5)	Proved	Thm. 4.1+C2+Lem. 4.3	Standard
Geom. cost (Lem. 4.4)	Proved	Construction	Standard
PRG (Thm. 4.7)	Proved (ROM)	Lem. 4.6+Argon2id	ROM
IND-CCA2 (Thm. 4.8)	Proved (ROM)	Four-game hybrid	ROM
Replay resistance (Thm. 5.1)	Proved	SUF-CMA+session id	Standard
Memory hardness	Proved	Alwen–Serbinenko [3]	Parallel ROM
Const.-time (partial)	Verified	5×10^5 samples, $< 1\%$	Empirical

Fix. Construction 2.2 generates $sid \xleftarrow{\$} \{0,1\}^{128}$ fresh at each encryption:

$$k_{MAC} \leftarrow H(k_0 \| MAC \| sid), \quad \tau \leftarrow \text{HMAC}(k_{MAC}, \text{hdr} \| sid \| \text{layers}).$$

The session identifier is stored in the file header (16 bytes overhead).

Theorem 5.1 (Replay resistance). *Under Construction 2.2, if HMAC-SHA3-256 is SUF-CMA secure, then for any PPT adversary $\mathcal{A}$ and any two distinct sessions $sid_1 \neq sid_2$:*

$$\Pr[\mathcal{A} \text{ replays } C^{sid_1} \text{ as valid in session } sid_2] \leq \varepsilon_{\text{SUF-CMA}} = \text{negl}(\kappa).$$

Proof. In session sid_2 , the verifier derives $k'_{MAC} = H(k_0 \| MAC \| sid_2)$. Since $sid_2 \neq sid_1$ and H is collision-resistant, $k'_{MAC} \neq k_{MAC}$ with overwhelming probability. Therefore $\text{HMAC}(k'_{MAC}, \text{hdr} \| sid_2 \| \text{layers}) \neq \tau$ with probability $1 - \varepsilon_{\text{SUF-CMA}}$. $\square$

5.2 OP6: Constant-Time Comparison (Partial)

The full pipeline analysis is in [2], Appendix C. We resolve components CT2a and CT2b.

CT2a — Magic-prefix comparison. The original comparison `plaintext[:5] == MFSU\x04` uses Python’s early-exit byte comparison, leaking the length of the common prefix via timing.

Fix:

```

1 import hmac as _hmac
2 MAGIC = b'MFSU\x04'
3
4 def check_magic(plaintext: bytes) -> bool:
5     return _hmac.compare_digest(plaintext[:5], MAGIC)
```

Listing 1: Constant-time magic-prefix check (CT2a).

CT2b — HMAC tag verification. Direct tag comparison with `==` is replaced by `hmac.compare_digest`.

Proposition 5.2 (CT4 timing uniformity). *Under the fixed implementation, the median decryption timing is statistically uniform across correct password, wrong-first-byte, and all-wrong-bytes cases.*

Residual open sub-problem. The KDF scratchpad access pattern remains non-constant-time. An oblivious-read replacement eliminates the cache-timing leak at $256 \times$ Phase 2 overhead [2]. Formalising the partial-leak security model is deferred (Open Problem OP2.).

Table 3: CT4 timing measurements ($n = 5 \times 10^5$ per case). Spread = (max – min)/mean of the three medians. Pass criterion: spread < 2% [2].

Component	Case	Median (ns)	Spread	CT4
CT2a — Magic prefix	Correct prefix	189	0.53%	Pass
	Wrong 1st byte	189		
	All wrong	190		
CT2b — Tag verify	Correct tag	154	0.65%	Pass
	Wrong 1st byte	154		
	All wrong	153		

6 Correction to Lemma 5.4 (Table 1)

The original preprint [1], Table 1, listed attacker ratios of $3.5\times, 7.5\times, 15.5\times$ for Levels 1, 2, 3 respectively. These values are inconsistent with Lemma 5.4 of the same document:

$$C_{\text{att}}(\ell) = C_{\text{base}} \cdot (2^{N(\ell)} - 1),$$

which yields $7\times, 15\times, 31\times$ for $N = 3, 4, 5$. The discrepancy appears to be a typographic error from an earlier draft in which the cost formula used a different normalisation. Table 1 of this paper uses the corrected values, consistent with Lemma 4.4.

Measured vs. theoretical ratios. The measured ratios ($6.4\times / 13.2\times / 26.8\times$) are lower than the theoretical bounds ($7\times / 15\times / 31\times$). This is expected: Argon2id with $p = 4$ lanes internally distributes work across parallel threads. When the time cost doubles, the total memory *work* doubles, but wall-clock time may increase by less than $2\times$ because lanes partially overlap execution. The security argument of Lemma 4.4 rests on memory work ($\Omega(m)$ per thread, C4), not wall-clock time. The measured ratios are a conservative lower bound on the security ratio.

7 Empirical Results

All measurements: reference Python implementation, `argon2-cffi` 25.1.0, single CPU thread (Python 3.12, Linux). Argon2id parameters: $t = 3$, $m = 65536$ KB, $p = 4$.

7.1 Geometric Cost Escalation

Table 4: Per-layer KDF timing (2 runs/layer, minimum).

Layer i	Time cost t	Time (s)	Ratio vs. Layer 0
0	3	0.171	1.00×
1	6	0.315	1.85×
2	12	0.606	3.55×
3	24	1.168	6.84×
4	48	2.316	13.56×

7.2 C1–C4 Conformance

7.3 OFV Security Game

Following [1], Appendix C:

Table 5: C1–C4 conformance checklist (criteria from [2], Section 6).

Check	Result	Criterion	Pass
V1 — Avalanche C1	$\mu = 0.4992, \sigma = 0.0027$	$[0.490, 0.510], \leq 0.010$	✓
V3 — Cost monotonicity C3	Strictly increasing ($t \in \{3, 6, 12, 24\}$)	Strictly increasing	✓
V4 — Memory hardness C4	64 MB/thread, ≤ 375 threads RTX 4090	Proved (A-S 2015)	✓
V5 — Roundtrip L1/L2/L3	Exact match	Exact match	✓
V5 — Wrong password	ValueError , no oracle signal	No oracle	✓
V5 — Tamper detection	ValueError (HMAC)	Reject	✓

- $C_{\text{base}} = 0.171 \text{ s}$ (Layer 0, $t = 3$).
- $C_{\text{att}}(\text{Level } 1) = 1.092 \text{ s}$ ($6.4 \times C_{\text{base}}$ measured; theory $7\times$).
- Wrong password queries: **ValueError** immediately. No oracle signal of any kind.
- Legitimate user: 0.171 s ($1 \times C_{\text{base}}$, Layer 0 only).

The OFV property is confirmed empirically: the adversary cannot determine password correctness without paying $C_{\text{att}}(\ell)$ per attempt.

7.4 Storage–Computation Asymmetry

Table 6: Storage–computation asymmetry (1 KB message, theoretical Argon2id RAM = $N \times 64 \text{ MB}$).

Level	Defender storage	Attacker RAM/attempt	Ratio
1	3,196 B	192 MB	62,993×
2	4,237 B	256 MB	63,355×
3	5,278 B	320 MB	63,574×

7.5 Test Vectors

Fixed inputs: password = `FractalShield_TestVector_v1`, salt = `0x000102...0f` (16 B), IV = `0x101112...1f` (16 B), message = `Hello, Abyss.` (13 B).

Table 7: Reference test vectors (deterministic).

Vector	Expected result
V1: KDF output ($t = 3$, 96 B)	<code>481e11f3cce157d6b7b2fd9b...</code>
V2: KDF ($t = 6$), diffBits vs V1	$\approx 50\%$ ✓
V3: Keystream (64 B, SHA3-256 CTR)	Deterministic given V1, IV
V4: PKCS7 padded + MAGIC check	correct: True; wrong: False ✓
V5: Roundtrip L1/L2/L3	All correct ✓

Full deterministic vectors generated by `fractalshield_argon2id.py`
`--vectors` at the project repository.

8 Anticipated Reviewer Questions

Q1. If the adversary knows the message distribution, could they distinguish the real layer statistically?

Answer: No, under C2.

The real layer is $CT_0 = \text{KS}(k_0, iv_0, L) \oplus \text{padded}$, where $\text{KS}(k_0, iv_0, L) \approx_c U_L$ (Theorem 4.7). Therefore $CT_0 \approx_c U_L$ regardless of the distribution of *padded*. XOR with a pseudorandom keystream completely masks the plaintext distribution, even if the adversary knows the message space. Decoy layers are masked identically (Theorem 4.8, game hop $G_2 \rightarrow G_3$, bound $\varepsilon_{\text{PRG}} + N/2^{40}$).

Q2. Measured ratios are lower than theoretical bounds. Does this invalidate Lemma 4.4?

Answer: No. Lemma 4.4 is a lower bound on memory work, not a wall-clock time prediction.

Argon2id with $p = 4$ lanes internally distributes work; doubling time cost doubles memory work exactly but may increase wall-clock time by less than $2\times$. The security argument rests on memory work ($\Omega(m)$ per thread, C4, Definition 2.1), not wall-clock time. Measured ratios are a conservative lower bound on the actual security ratio. See Section 6.

Q3. The IND-CCA2 proof uses the ROM. Is the construction secure without it?

Answer: Unknown; explicitly stated as Open Problem OP1..

Theorems 4.1, 4.2, 4.5, and 5.1 are proved in the *standard model*. Only Theorems 4.7 and 4.8 require the ROM, specifically for the SHA3-256 counter-mode keystream. A standard-model reduction would require substituting the keystream with a construction having a formal standard-model proof (e.g., AES-CTR with a concrete hardness reduction), which does not affect any other part of the construction. We do not claim standard-model security; the limitation is stated in Section .

Q4. Could a timing side-channel on decryption reveal which layer is real, breaking OFV?

Answer: Partially mitigated; the residual is a stated open problem.

Two of four timing side-channels are resolved (Section 5): CT2a (magic-prefix comparison, spread 0.53%) and CT2b (MAC tag comparison, spread 0.65%). The residual is the Argon2id scratchpad access pattern, which is data-dependent in Argon2id mode. An oblivious-read fix is described in [2] at $256\times$ Phase 2 overhead.

Remark 8.1 (Scope of OFV). Theorem 4.5 holds in the standard offline threat model (Definition 5.1: no physical access to the decryption machine). Against a local timing adversary (Definition 9.7 of [2]), the OFV guarantee holds for CT2a and CT2b (verified empirically) but not yet for the scratchpad access pattern. This is consistent with the state of the art: no published memory-hard KDF provides a fully constant-time guarantee for the data-dependent phase.

9 Related Work

Memory-hard KDFs. `scrypt` [8] introduced sequential memory-hardness. Argon2 [4] formalised the parallel ROM model. Both slow each attempt but preserve the MAC verification oracle. FractalShield-Argon2id uses Argon2id as core F and eliminates the oracle structurally.

Honey Encryption. Juels and Ristenpart [7] produce plausible plaintexts for wrong keys via a distribution-transforming encoding (DTE). This reduces the adversary’s confidence in guesses but provides neither structural oracle elimination nor geometric cost escalation, and requires a

DTE tailored to each message space. FractalShield-Argon2id’s OFV is structural and message-space agnostic. The two constructions operate in different design regimes and are complementary (see Table 8).

Table 8: Design regime comparison: Honey Encryption vs. FractalShield-Argon2id.

Property	Honey Enc. [7]	FS-Argon2id (this)
Oracle elimination mechanism	Semantic confusion (DTE)	Structural (full KDF cost)
Message-space dependence	Yes (DTE per space)	No (agnostic)
False positives (wrong key)	Yes (by design)	$< N/2^{40}$ (Lem. 4.3)
Attacker cost/attempt	$1 \times \text{KDF}$	$(2^N - 1) \times \text{KDF}$
Attacker RAM/attempt	$O(1)$	$\geq N \times 64 \text{ MB}$
Geometric cost escalation	No	Yes (Lem. 4.4)
Replay resistance	No	Yes (Thm. 5.1)

Remark 9.1 (Complementarity). Honey Encryption and FractalShield-Argon2id can be composed: the DTE is applied to the plaintext before encryption, adding semantic confusion on top of structural oracle elimination. The two mechanisms are independent and additive.

Table 9: FractalShield-Argon2id vs. existing constructions.

Property	AES-GCM	Ar2id+AES	Honey Enc.	FS-Ar2id (ours)
IND-CCA2	Yes	Partial	Yes	Yes (ROM)
Memory-hard KDF	Ext.	Yes	No	Yes (proved)
No verification oracle	No	No	Partial	Yes
Geometric cost escalation	No	No	No	Yes
Layer indistinguishability	N/A	N/A	Partial	Yes
Message-space agnostic	Yes	Yes	No	Yes
Replay resistance	No	No	No	Yes
Serverless offline	Yes	Yes	Yes	Yes
Public cryptanalysis	25+ yr	10+ yr	10+ yr	None yet

Full comparison. We note explicitly that “public cryptanalysis: none yet” is a real limitation (Open Problem [OP5.](#)) and not a claim of security beyond what the formal proofs establish.

10 Open Problems

OP1. IND-CCA2 without ROM. Theorems 4.7 and 4.8 assume the ROM. A standard-model reduction to a concrete hardness assumption is desirable for high-assurance deployments.

OP2. Scratchpad oblivious read (OP6 residual). The Argon2id scratchpad access pattern in Phase 2 is data-dependent. An oblivious-read replacement eliminates the cache-timing leak at $256 \times$ Phase 2 overhead [2]. Formalising the partial-leak security model is deferred.

- OP3. Password entropy bound (OP8).** A lower bound $H_\infty(pwd) \geq f(\ell, T)$ as a function of protection level ℓ and attacker budget T is needed to make the security guarantees concrete for low-entropy passwords.
- OP4. Quantum security.** Grover’s algorithm reduces exhaustive search from $|\mathcal{D}|$ to $\sqrt{|\mathcal{D}|}$ queries. The geometric escalation factor $(2^N - 1)$ is *preserved* under Grover: each quantum oracle query still costs $C_{\text{att}}(\ell)$ quantum gates (Lemma 9.4 of [2]). A full QROM analysis is deferred.
- OP5. Independent cryptanalysis.** No external security audit has been performed on this construction. We invite public cryptanalysis at <https://github.com/Fracta-Axis/-Fractalyx-Argon2id>.
- OP6. Mechanised verification.** A formalisation in EasyCrypt or CryptoVerif would provide higher assurance for the four-game IND-CCA2 hybrid (Theorem 4.8).

11 Conclusion

We presented FractalShield-Argon2id, a complete, self-contained instantiation of the FractalShield oracle-free layered encryption framework using Argon2id as the cryptographic core. The construction inherits four formal security theorems (Integrity, IV Uniqueness, OFV, IND-CCA2), adds a formal proof of replay resistance (resolving OP3), and provides partial constant-time verification (resolving OP6 for two of four pipeline components).

The choice of Argon2id eliminates the main unresolved dependencies of the MFSU-Crypt instantiation by substituting a primitive with a decade of public cryptanalysis and proved memory-hardness under the parallel ROM. The geometric cost escalation ($7 \times /15 \times /31 \times$), oracle-free verification, and storage–computation asymmetry ($\geq 62,000 \times$ for 1 KB) are all verified empirically.

FractalShield-Argon2id is, to our knowledge, the first documented password-based encryption scheme simultaneously achieving: no verification oracle, geometric cost escalation, layer statistical indistinguishability, replay resistance, and serverless offline operation—with all properties backed by formal proofs or empirical measurements, and built entirely on standardised, publicly audited primitives.

We invite the community to propose new core instantiations, run the conformance checklist of [2], and contribute to the open problems above.

Attribution. The oracle-free verification through layered magic-prefix detection combined with geometric cost escalation was first introduced by Miguel Angel Franco León in [1].

Acknowledgements. The author thanks the open cryptography community for the public codebases and papers that made this work possible.

References

- [1] M. A. Franco León. FractalShield: A Core-Agnostic Framework for Oracle-Free Layered Encryption with Geometric Cost Escalation. Preprint v1.0, May 2026. <https://github.com/Fracta-Axis/Fractalyx>
- [2] M. A. Franco León. Technical Note: The Four Core Laws of FractalShield — A Framework for Oracle-Free Verification and Geometric Cost Escalation v1.2. Fracta-Axis Project, May 2026. <https://github.com/Fracta-Axis/Fractalyx>

- [3] J. Alwen and V. Serbinenko. High parallel complexity graphs and memory-hard functions. In *Proc. STOC 2015*, pp. 595–603, 2015.
- [4] A. Biryukov and D. Khovratovich. Argon2: New generation of memory-hard functions. In *IEEE EuroS&P 2016*, pp. 292–302, 2016.
- [5] A. Biryukov, D. Khovratovich, and S. Josefsson. Argon2 memory-hard function for password hashing and proof-of-work applications. RFC 9106, IETF, September 2021.
- [6] M. Bellare and P. Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *Proc. CCS 1993*, pp. 62–73, 1993.
- [7] A. Juels and T. Ristenpart. Honey encryption: Security beyond the brute-force bound. In *EUROCRYPT 2014*, LNCS 8441, pp. 293–310, 2014.
- [8] C. Percival. Stronger key derivation via sequential memory-hard functions. In *BSDCan 2009*, 2009.
- [9] J.-P. Aumasson, S. Neves, Z. Wilcox-O’Hearn, and C. Winnerlein. BLAKE2: simpler, smaller, fast as MD5. In *ACNS 2013*, LNCS 7954, pp. 119–135, 2013.
- [10] NIST. Recommendation for Password-Based Key Derivation. SP 800-132, 2010.
- [11] J. Blocki, B. Harsha, and S. Zhou. On the computational complexity of minimal cumulative cost graph pebbling. In *ITCS 2018*, 2018.